2026-11-02 로그인 수정 정리

- 대상: `lgcns-fe-next`, Next.js 16.4.3, React 19.3.0, Prisma 6.19.3, MongoDB
- 기준은 이 날짜에 저장소에 남은 소스다. 화면의 로그인 제출은 네트워크로 자기 API를 다시 부르지 않는다. 페이지가 로그인 라우트의 `POST` 함수를 호출하고, 그 함수가 `userService.login`을 호출한다.
- 데이터베이스 비밀번호와 전체 연결 문자열은 적지 않는다. 값은 `lgcns-fe-next/.env.local`의 `DATABASE_URL`에만 있다.

1. 오늘 바뀐 일

1. 원격 로그인 서버 호출을 페이지에서 빼냈다. 현재 `app/(auth)/login/page.tsx`에는 그 주소가 없다.
2. Prisma와 MongoDB를 연결했다. 스키마의 모델은 `User`다. 접속 데이터베이스 이름은 `lgcns-7th`다. 클러스터 호스트는 `spharos.kz3esds.mongodb.net`다.
3. `service/userService.ts`가 사용자 목록, 생성, 로그인을 담당한다. 비밀번호는 `scryptSync`로 해시해 `소금:해시` 문자열로 저장한다.
4. `POST /api/user`로 사용자를 만들고, `POST /api/auth/login`이 그 사용자를 확인한다. 두 라우트 모두 `userService`만 호출한다.
5. 로그인 페이지의 서버 함수는 라우트의 `POST`를 함수로 호출한다. 성공하면 `accessToken` 쿠키를 심고 `redirect("/")` 한다. 실패하면 `redirect("/login")` 한다.
6. 점검용 계정 `test@example.com` / `password12`, 이름 테스트를 `lgcns-7th`의 `User` 컬렉션에 넣었다.

2. 설치와 기본 설정

작업 디렉터리는 `lgcns-fe-next`다. `package.json`에 `engines`는 없다. `prisma`와 `@prisma/client`는 `dependencies`에 `^6.19.3`으로 있다.

```
cd lgcns-fe-next
npm install
npx prisma generate
```

`npx prisma generate`는 `prisma/schema.prisma`를 읽고 `node_modules/@prisma/client`를 만든다. 데이터베이스에 접속하지 않는다.

컬렉션을 맞출 때는 Prisma CLI가 `.env`만 읽고 `.env.local`은 읽지 않으므로, 셸에서 변수를 먼저 올린다.

```
set -a && source .env.local && set +a
npx prisma db push
```

이 명령이 lgcns-7th에 컬렉션 User와 이메일 고유 인덱스 User_email_key를 만들었다. 개발 서버는 npm run dev다. Next.js는 프로세스가 시작할 때 .env.local을 읽는다. 연결 문자열을 바꾼 뒤에는 서버를 다시 시작한다.

DATABASE_URL의 형태는 이렇다. 사용자, 비밀번호, 호스트는 실제 값으로 바꾼다. 경로의 데이터베이스 이름은 lgcns-7th다.

```
DATABASE_URL="mongodb+srv://USER:PASSWORD@HOST/lgcns-7th?retryWrites=true&w=majority"
```

.gitignore의 .env*가 이 파일을 커밋에서 제외한다. schema.prisma는 url = env("DATABASE_URL")로만 주소를 받는다.

스키마 선언	의미
provider = "mongodb"	접속 대상이 MongoDB
id String @id @default(auto()) @map("_id") @db.ObjectId	문서 _id를 Prisma의 id로 사용
email String @unique	이메일 고유 인덱스
password String	평문이 아닌 소금:해시
createdAt DateTime @default(now())	생성 시각
updatedAt DateTime @updatedAt	수정 시각

lib/prisma.ts는 PrismaClient를 하나 만들어 prisma로 내보낸다. 개발 모드에서는 globalThis에 같은 인스턴스를 둔다. 이 파일에는 "use client"가 없다.

3. 파일과 선언

파일	선언	모듈	하는 일
app/(auth)/login/page.tsx	async function handleSubmit 안의 "use server"	서버 컴포넌트	폼 값을 읽고 로그인 POST를 호출한 뒤 쿠키와 redirect를 처리
component/templates/LoginSection.tsx	function LoginSection	서버 컴포넌트	<form action={handleSubmit}>
component/organisms/LoginForm.tsx	파일 첫 줄 "use client"	클라이언트 컴포넌트	입력 상태, Zod 검사, useFormStatus, 로그인 상태면 /로 이동
app/api/auth/login/route.ts	export async function POST	라우트 핸들러	JSON을 검사하고 userService.login을 호출
app/api/user/route.ts	export async function GET, POST	라우트 핸들러	userService.list, userService.create

파일	선언	모듈	하는 일
service/userService.ts	export const userService	서버 모듈	해시, 검증, 목록, 생성, 로그인
lib/prisma.ts	export const prisma	서버 모듈	PrismaClient 한 개
prisma/schema.prisma	model User	Prisma 스키마	MongoDB 컬렉션 User
component/providers/authCookie.ts	export const ACCESS_TOKEN_COOKIE	모듈	쿠키 이름 "accessToken"
component/providers/AuthShell.tsx	async function AuthShell	서버 컴포넌트	cookies()로 토큰을 읽어 AuthProvider에 넘김
app/layout.tsx	function RootLayout	서버 컴포넌트	Suspense fallback={null} 안에 AuthShell

(auth)는 URL에 없는 라우트 그룹이다. 화면 주소는 /login이다. HTTP로 직접 호출할 수 있는 주소는 /api/auth/login과 /api/user다.

"use server"는 handleSubmit을 서버 함수로 표시하는 지시문이다. 함수 본문의 첫 문장이다. "use client"는 그 파일이 클라이언트 컴포넌트 그래프의 시작이라는 지시문이다. 둘 다 호이스팅 문법이 아니다.

handleSubmit과 라우트의 POST에 붙은 async는 ECMAScript async 함수다. 호출 결과가 프로미스다. HTML의 <script async>와 같은 속성이 아니다.

4. 로딩 시점

이 파일들은 HTML에 <script src>로 적혀 있지 않다. Next가 서버에서 서버 컴포넌트와 라우트 핸들러를 실행하고, "use client"로 시작한 그래프만 브라우저 모듈로 보낸다. Prisma와 node:crypto 호출은 그 브라우저 모듈에 없다.

코드	적힌 곳	로딩
<script src="a.js"></script>	HTML. async와 defer가 없음	동기. 파서가 그 자리에서 멈추고, 받고, 실행한 뒤 파싱을 재개한다
<script async src="a.js"></script>	HTML	받는 즉시 실행한다. 여러 스크립트의 실행 순서는 문서 순서가 아니다
<script defer src="a.js"></script>	HTML	문서를 파싱한 뒤, 문서에 나온 순서로 실행한다
<script type="module" src="a.js"></script>	HTML	모듈 그래프. async가 없으면 defer처럼 파싱이 끝난 뒤 실행한다

코드	적힌 곳	로딩
`async function handleSubmit`	`login/page.tsx`	스크립트 로딩 속성이 아니다. 호출될 때 프로미스를 반환한다. 실행 위치는 Next 서버다
`export async function POST`	`app/api/auth/login/route.ts`	HTTP `POST`가 오거나, 페이지가 이 함수를 직접 호출할 때 서버에서 실행된다
`"use client"` 파일	`LoginForm.tsx`	Next가 브라우저용 모듈로 보낸다. 손글씨 `<script>`는 없다

HTML 명세에서 고전 스크립트에 `async`도 `defer`도 없으면 파서를 막는다. `defer`는 문서 파싱이 끝난 뒤 순서대로 실행한다. `type="module"`인 외부 스크립트는 `async` 속성이 없을 때 지연 실행된다. ECMA-262의 async 함수는 그와 별개로, 호출 결과를 Promise로 감싼다.

5. 로그인 흐름

1. `LoginSection`이 서버 컴포넌트에서 `<form action={handleSubmit}>`을 그린다. 서버 HTML의 폼은 `method="POST"`이고, 숨은 입력에 서버 액션 식별자가 있다.
2. 폼 안의 `LoginForm`은 클라이언트 컴포넌트다. 제출되는 값은 React 상태가 아니라 `name="userId"`, `name="password"`의 `FormData`다. 제출을 기다리는 동안 버튼 문구는 로그인중이다. `useFormStatus`는 그 폼의 자식에서만 `pending`을 본다.
3. `handleSubmit`이 `userId`와 `password`를 읽는다. `FormData.get`의 반환은 `string | File | null`이다. 둘 중 하나라도 문자열이 아니면 `redirect("/login")` 한다.
4. 페이지는 `POST as login`으로 가져온 라우트 함수를 호출한다. 인자는 `new Request("http://localhost/api/auth/login", { method: "POST", headers: { "Content-Type": "application/json" }, body })`다. 이 호출은 소켓을 열지 않는다. 같은 함수가 HTTP `POST /api/auth/login`의 핸들러이기도 하다.
5. 라우트가 `request.json()`으로 본문을 읽고 Zod로 `email`, `password`가 비어 있지 않은지 본다. 실패하면 상태 `400`이다.
6. `userService.login`이 이메일로 `prisma.user.findUnique`를 한다. 없거나 `verifyPassword`가 거짓이면 `{ ok: false, message: "이메일 또는 비밀번호가 올바르지 않습니다." }`다. 라우트는 그때 상태 `401`을 반환한다. 없는 사용자와 틀린 비밀번호를 문구로 구분하지 않는다.
7. `verifyPassword`는 저장된 `소금:해시`를 나누고, 같은 소금으로 `scryptSync(password, salt, 64)`를 다시 계산한다. 비교는 `timingSafeEqual`이다. 길이가 다르면 비교하지 않고 거짓이다.
8. 맞으면 `randomBytes(32).toString("hex")`가 `accessToken`이 된다. `expiresIn`은 `7200`이다. 이 토큰은 MongoDB에 저장하지 않는다. JWT가 아니다.
9. 페이지는 `data.success`가 참이고 `accessToken`이 문자열일 때만 `cookies().set`을 한다. 쿠키 이름은 `accessToken`이다. `httpOnly`, `sameSite: "lax"`, `path: "/"`, `maxAge`는 응답의 `expiresIn` 또는 `7200`이다. `secure`는 `NODE_ENV`가 `production`일 때만 참이다. 그다음 `redirect("/")`다.
10. 성공이 아니면 `redirect("/login")`이다. 서버 함수가 값을 반환하고 같은 응답에서 로그인 페이지를 다시 그리면, `app/layout.tsx`의 `AuthShell`이 `cookies()`를 읽는 지점에서 Next가 `cookies`와

mutableCookies가 다르다는 오류를 냈다. redirect는 그 렌더 전에 응답을 끝낸다.

11. 다음 문서 요청에서 AuthShell이 쿠키를 읽어 AuthProvider에 넘긴다. isLoggedIn은 쿠키 문자열이 있는지만 본다. 토큰을 MongoDB나 서명으로 다시 확인하는 코드는 없다. 클라이언트 LoginForm은 isLoggedIn이 참이면 useEffect 안에서 router.replace("/")를 호출한다.

6. 사용자 만들기

컬렉션이 비어 있으면 로그인은 실패한다. 계정을 만드는 HTTP 경로는 POST /api/user다.

1. email은 이메일 형식, password는 8자 이상, name은 없어도 된다.
2. 같은 이메일이 있으면 409와 이미 등록된 이메일입니다.다.
3. hashPassword가 16바이트 소금을 16진수로 만들고 scryptSync로 64바이트 해시를 만든다.
4. prisma.user.create의 select에는 비밀번호가 없다. 성공 상태는 201이다.

GET /api/user는 findMany로 같은 공개 필드만 돌려준다.

오늘 넣은 점검 계정은 이메일 test@example.com, 비밀번호 password12, 이름 테스트다. Atlas에서 볼 위치는 데이터베이스 lgcns-7th, 컬렉션 User다. 컬렉션 이름은 users가 아니다. 이전 데이터베이스 spharos에 남아 있는 문서는 앱이 더 이상 읽지 않는다.

7. 확인한 결과

요청	결과
로그인 폼에 test@example.com / password12	303, Location: /, Set-Cookie: accessToken=...; HttpOnly; SameSite=lax; Max-Age=7200
로그인 폼에 다른 비밀번호	303, Location: /login. 쿠키를 심지 않음
브라우저에서 같은 계정으로 제출	버튼이 로그인중이 된 뒤 /로 이동하고 화면에 로그인됨
POST /api/user로 계정을 만든 직후 POST /api/auth/login	맞는 비밀번호는 200과 accessToken, 다른 비밀번호는 401

8. 공식 문서

- Node.js, scryptSync: https://nodejs.org/api/crypto.html#cryptoscryptsyncpassword-salt-keylen-options
- Node.js, timingSafeEqual: https://nodejs.org/api/crypto.html#cryptotimingsafeequala-b
- Node.js, randomBytes: https://nodejs.org/api/crypto.html#cryptorandombytessize-callback
- Prisma, MongoDB: https://www.prisma.io/docs/orm/overview/databases/mongodb
- Prisma, 연결 문자열: https://www.prisma.io/docs/orm/reference/connection-urls
- Prisma, db push: https://www.prisma.io/docs/orm/reference/prisma-cli-reference#db-push
- Next.js, 환경 변수와 .env.local: https://nextjs.org/docs/app/guides/environment-variables
- Next.js, 라우트 핸들러: https://nextjs.org/docs/app/getting-started/route-handlers

- Next.js, 서버 함수: https://nextjs.org/docs/app/getting-started/updating-data
- Next.js, `cookies`: https://nextjs.org/docs/app/api-reference/functions/cookies
- Next.js, `redirect`: https://nextjs.org/docs/app/api-reference/functions/redirect
- MDN, `FormData.get`: https://developer.mozilla.org/en-US/docs/Web/API/FormData/get
- MDN, `Request`: https://developer.mozilla.org/en-US/docs/Web/API/Request/Request
- Fetch 표준, 문자열 본문과 `Content-Type`: https://fetch.spec.whatwg.org/#concept-bodyinit-extract
- ECMA-262, async 함수: https://tc39.es/ecma262/multipage/ecmascript-language-functions-and-classes.html#sec-async-function-definitions
- HTML 명세, `script`의 `async`, `defer`, 모듈: https://html.spec.whatwg.org/multipage/scripting.html#the-script-element
- React, `useFormStatus`: https://react.dev/reference/react-dom/hooks/useFormStatus